

762. Prime Number of Set Bits in Binary Representation

Solved 

Easy

 Topics

 Companies

 Hint

Given two integers `left` and `right`, return the **count** of numbers in the **inclusive** range `[left, right]` having a **prime number of set bits** in their binary representation.

Recall that the **number of set bits** an integer has is the number of `1`'s present when written in binary.

- For example, `21` written in binary is `10101`, which has `3` set bits.

Example 1:

Input: `left = 6, right = 10`

Output: `4`

Explanation:

`6` -> `110` (2 set bits, 2 is prime)

`7` -> `111` (3 set bits, 3 is prime)

`8` -> `1000` (1 set bit, 1 is not prime)

`9` -> `1001` (2 set bits, 2 is prime)

`10` -> `1010` (2 set bits, 2 is prime)

4 numbers have a prime number of set bits.

Example 2:

Input: left = 10, right = 15

Output: 5

Explanation:

10 -> 1010 (2 set bits, 2 is prime)

11 -> 1011 (3 set bits, 3 is prime)

12 -> 1100 (2 set bits, 2 is prime)

13 -> 1101 (3 set bits, 3 is prime)

14 -> 1110 (3 set bits, 3 is prime)

15 -> 1111 (4 set bits, 4 is not prime)

5 numbers have a prime number of set bits.

Constraints:

- $1 \leq \text{left} \leq \text{right} \leq 10^6$
- $0 \leq \text{right} - \text{left} \leq 10^4$

Python:

class Solution:

```
def countPrimeSetBits(self, left: int, right: int) -> int:  
    count = 0
```

```
    for i in range(left, right + 1):  
        setBits = bin(i).count('1') # Convert to binary and count '1's  
        if self.isPrime(setBits):  
            count += 1
```

```
    return count
```

```
def isPrime(self, n: int) -> bool:
```

```
    if n <= 1:  
        return False  
    for i in range(2, int(n ** 0.5) + 1):  
        if n % i == 0:  
            return False
```

```
return True
```

JavaScript:

```
var countPrimeSetBits = function(left, right) {  
    let count = 0;  
  
    for (let i = left; i <= right; i++) {  
        let setBits = i.toString(2).split('1').length - 1; // Count 1's in binary  
        if (isPrime(setBits)) {  
            count++;  
        }  
    }  
  
    return count;  
};  
  
function isPrime(n) {  
    if (n <= 1) {  
        return false;  
    }  
    for (let i = 2; i * i <= n; i++) {  
        if (n % i === 0) {  
            return false;  
        }  
    }  
    return true;  
}
```

Java:

```
class Solution {  
    public int countPrimeSetBits(int left, int right) {  
        int count=0;  
  
        for (int i=left;i<=right;i++) {  
            int setBits=Integer.bitCount(i);  
            if (isPrime(setBits)) {  
                count++;  
            }  
        }  
        return count;  
}  
  
private boolean isPrime(int n) {  
    if (n<=1) {  
        return false;  
    }  
}
```

```
    }  
    for (int i=2;i*i <= n;i++) {  
        if (n%i==0) {  
            return false;  
        }  
    }  
  
    return true;  
}  
}
```